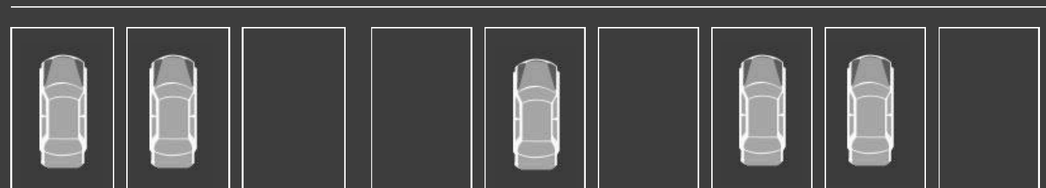
Smart guard

MEET SMART GUARD

Welcome to the future of parking where technology meets convenience, and every space counts.

an IoT parking system with an auto gate feature that regulates entry based on spot capacity. It enables the guard to effectively monitor and manage spot availability.



PURPOSE OF THE PROJECT

01

TIME MANAGEMENT

Email is being sent to the security guard when the parking's are full.

02

AUTOMATED MONITOR

Monitor the system and how many cars are in or out.

03

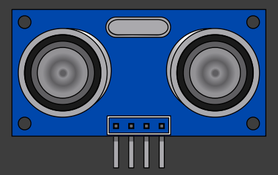
MORE SECURE

avoid any traffic that may occur during rush hour.

COMPONENTS

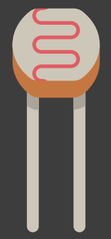
ESP32

The ESP32 helps to know the status of the parking cars, via the dashboard.



Ultrasonic

Ultrasonic sensors accurately measure the distance between the sensor and the car .



LDR

LDRs can be used to detect the presence of cars in parking spaces.

CODE

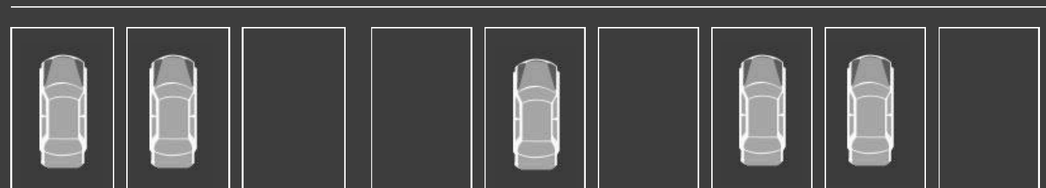
```
#include <ESP32Servo.h>
#include <ThingSpeak.h>
#include <WiFi.h>
//Constans variables intialization
#define SOUND_SPEED 0.034
// ThingSpeak
#define CHANNEL_ID 2520072
#define CHANNEL_API_KEY "QMRPB94D64XHOBVU"
//WIFI
#define WIFI_TIMEOUT_MS 20000
#define WIFI_NETWORK "E5576_8A0B"
#define WIFI_PASSWORD "35664187136"
```

```
// count number of cars
int counter =0;
//LDR Sensor Pins intialization
int sensorP1Pin = 33;
int sensorP2Pin = 34;
int sensorP3Pin = 35;

//Ultrasonic Sensor Pins intialization
int trigPin = 27;
int echoPin = 26;
long duration;
float distanceCm;
```

```
//InfraRed Sensor Pin intialization
int IRPin = 32;
//LEDs Pin intialization
int led1Pin = 23;
int led2Pin = 18;
int led3Pin = 19;
//Buzzer Pin intialization
int buzzerPin = 5;
// servo object intialization
Servo myservo;
int servoPin = 25;

WiFiClient Client ;
```



CODE

```
// wifi connection function
void connectToWiFi(){
  Serial.print("connecting to wifi");
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_NETWORK,WIFI_PASSWORD);
  unsigned long startAttemptTime = millis();

  while( (WiFi.status()!= WL_CONNECTED )&& ((millis()- startAttemptTime) <
  WIFI_TIMEOUT_MS)) {
    Serial.print(".");
    delay(100);
  }
  if (WiFi.status() != WL_CONNECTED ){
    Serial.println("failed");

  }
  else{
    Serial.print("Connected");
    Serial.println(WiFi.localIP());
  }
}
```

CODE

```
void setup() {
Serial.begin(115200);
//To start WIFI connection
connectToWiFi();
//To start ThingSpeak connection
ThingSpeak.begin(Client);

pinMode(sensorP1Pin,INPUT);
pinMode(sensorP2Pin,INPUT);
pinMode(sensorP3Pin,INPUT);

myservo.attach(servoPin);

pinMode(led1Pin, OUTPUT);
pinMode(led2Pin, OUTPUT);
pinMode(led3Pin, OUTPUT);

pinMode(trigPin, OUTPUT);
pinMode(echoPin, INPUT);

pinMode(buzzerPin, OUTPUT);

pinMode(IRPin,INPUT);
// Setting an initial state of Servo
myservo.write(60);
}
```

CODE

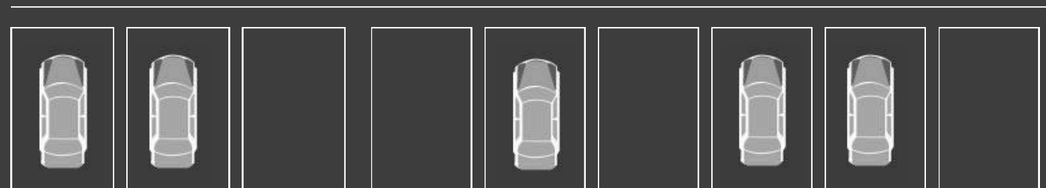
```
void loop() {
//reading values from sensors
  int sensorP1Value = analogRead(sensorP1Pin);
  int sensorP2Value = analogRead(sensorP2Pin);
  int sensorP3Value = analogRead(sensorP3Pin);

  int IRvalue = digitalRead(IRPin);

  float distanceCm = ultrasonic_read();
  //calculate the LDR sensors sum
  int parkingSum = sensorP1Value + sensorP2Value + sensorP3Value;

  //make a single http request to send the data
 ThingSpeak.writeField(CHANNEL_ID,1,parkingSum, CHANNEL_API_KEY);
  Serial.println(counter);
```

```
  // to contorl the LEDs in the parking
 if(sensorP1Value <70  ){
  digitalWrite(led1Pin,HIGH);
 }
 else{
  digitalWrite(led1Pin,LOW);
 }
if(sensorP2Value <70  ){
  digitalWrite(led2Pin,HIGH);
 }
 else{
  digitalWrite(led2Pin,LOW);
 }
 if(sensorP3Value <70  ){
  digitalWrite(led3Pin,HIGH);
 }
 else{
  digitalWrite(led3Pin,LOW);
 }
```



CODE

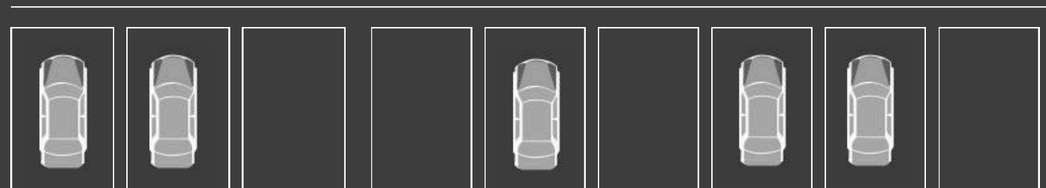
```
//to control the parking lot entry max. 3 cars
 if((distanceCm <= 3 && parkingSum <= 600) || counter > 3) {
 myservo.write(60); //gate close
 counter=0;
 digitalWrite(buzzerPin, HIGH);
 delay(1000);
 digitalWrite(buzzerPin, LOW);
}
 else if (distanceCm <= 3){
  counter++;
    myservo.write(110);
    delay(3000);
     myservo.write(60);
 }
 ////to control the parking lot exit
  if(IRvalue == 0){
   Serial.println("Someone is here");
   myservo.write(110);
   delay(3000);
   myservo.write(60);
  } }
```

```
//Ultrasonic function to read and calculate distance
 long ultrasonic_read(){

  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);

  // Reads the echoPin, returns the sound wave travel time in microseconds
  duration = pulseIn(echoPin, HIGH);

  // Calculate the distance
  distanceCm = duration * SOUND_SPEED/2;
  return distanceCm;
 }
```



NOTIFICATION

9:59 م (قبل 29 دقيقة)

<action@ifttt.com> **Webhooks via IFTTT**

أنا

الترجمة إلى العربية

What: PARKING_LOT_STATUS
When: April 27, 2024 at 09:59PM
Extra Data: There is no parking spots available

Manage

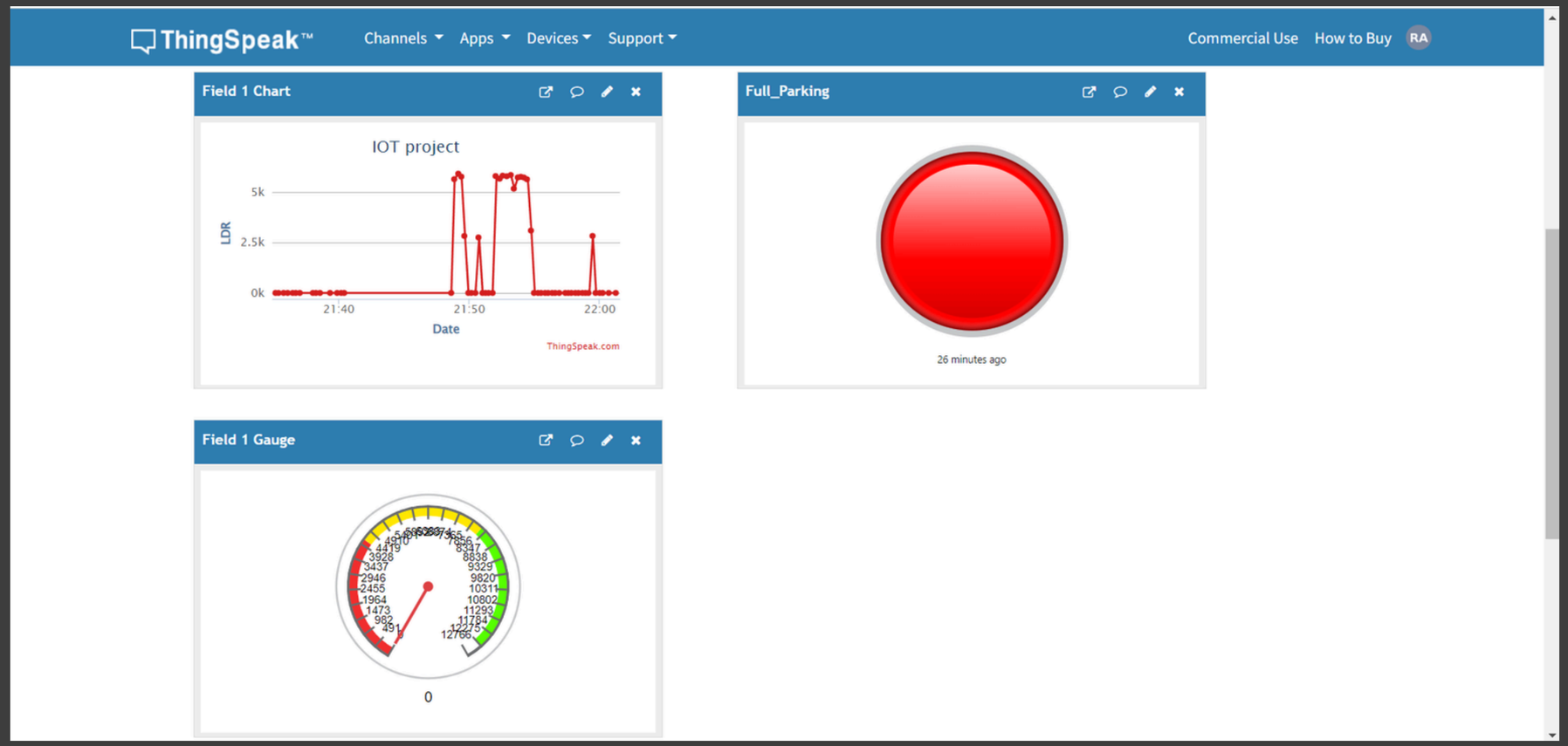
Unsubscribe from these notifications or sign in to manage your Email service.

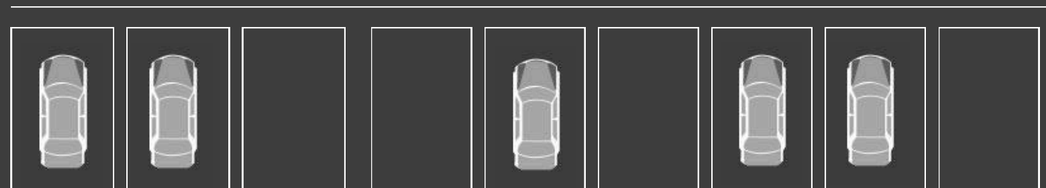
IFTTT

Download on the App Store

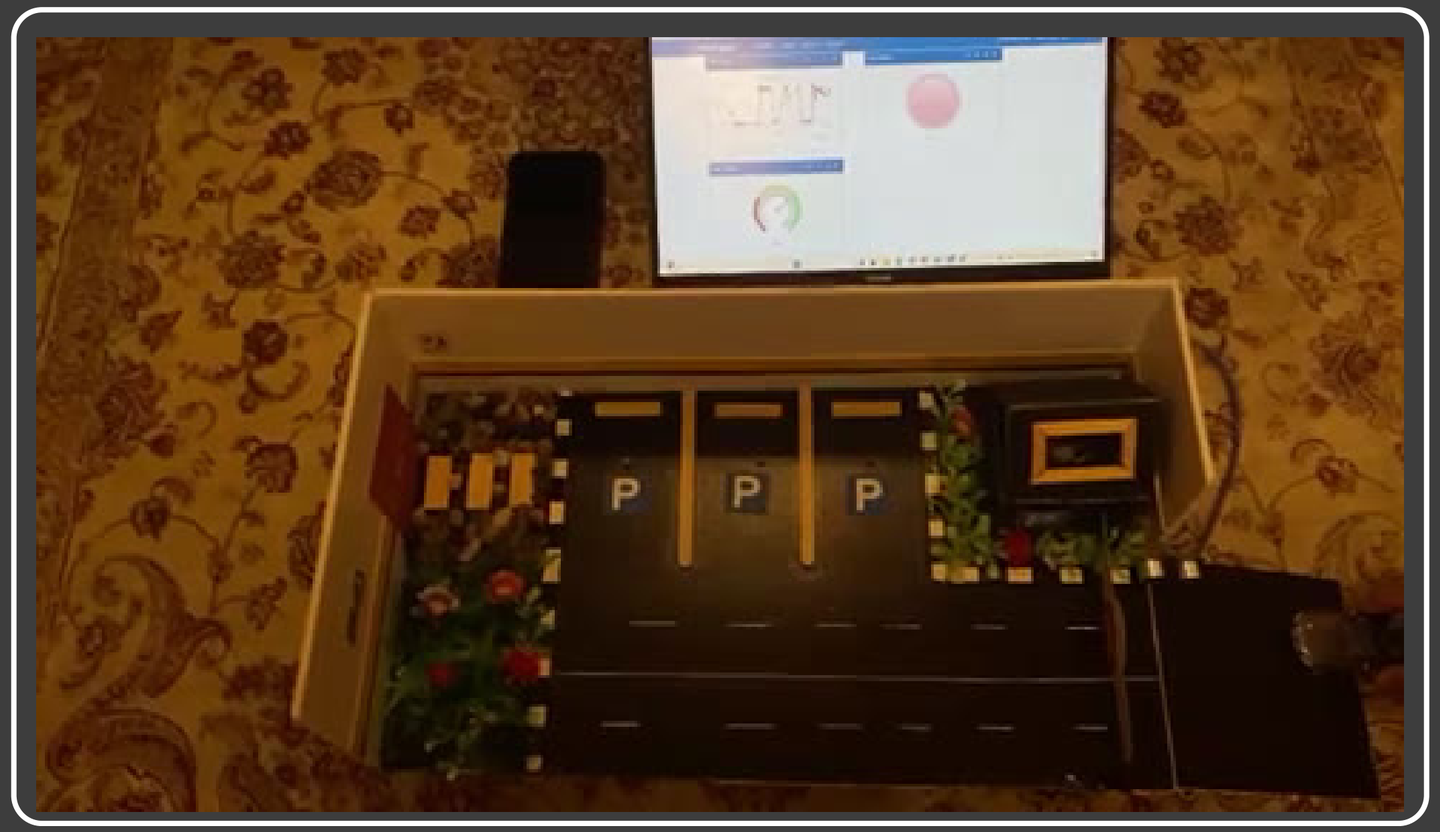
GET IT ON Google Play

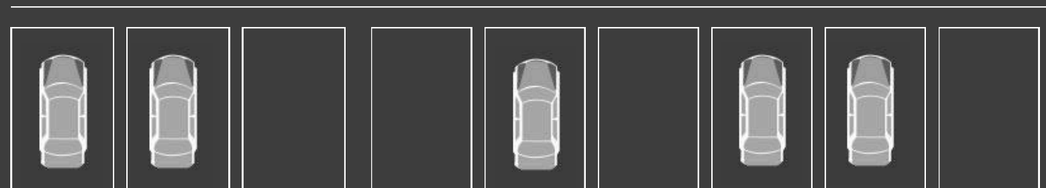
DASHBOARD

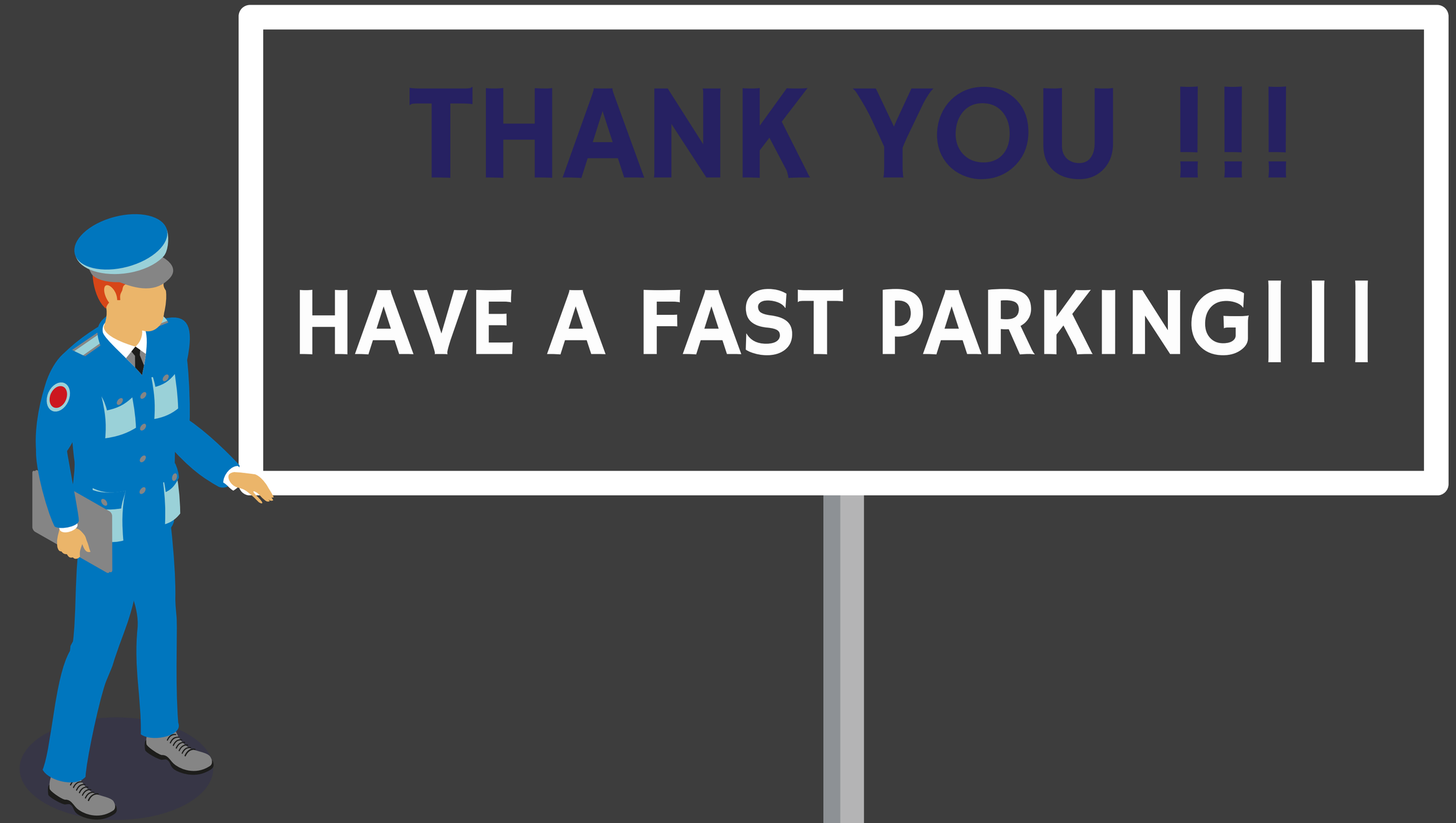

ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
RA
Field 1 Chart
IOT project
5k
2.5k
0k
LDR
21:40
21:50
22:00
Date
ThingSpeak.com
Full_Parking
26 minutes ago
Field 1 Gauge
0



LET’S PARK






THANK YOU !!!
HAVE A FAST PARKING|||